



Events & Interactive Applications

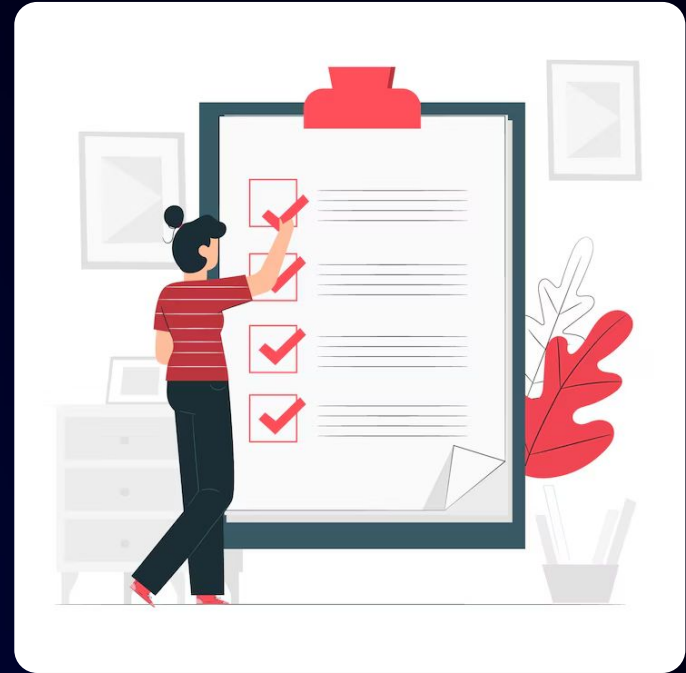
Learning Objectives

- Understand the basics of events in JavaScript and their role in web interactivity.
- Explore various event types, including keyboard and mouse events.
- Learn how to add and remove event listeners to handle events dynamically.
- Grasp the concepts of event propagation, including bubbling and capturing phases.
- Implement event delegation for efficient event handling.
- Master techniques to prevent default behaviors and control event propagation.



Prerequisites

- Strong understanding of HTML structure and DOM elements.
- Basic knowledge of JavaScript fundamentals (variables, functions, conditions, loops).
- Familiarity with DOM selection methods like `querySelector` and `getElementById`.
- Understanding of basic DOM manipulation (changing content, attributes, and styles).
- Awareness of browser behavior and how web pages respond to user actions (clicks, typing, etc.).





Introduction to JavaScript Events & Types

JavaScript Events

- **Definition:** Events are actions or occurrences that happen in the browser due to user interaction or system changes.
- **Importance:**
 - **Interactivity:** Events allow web pages to respond to user actions.
 - **User Experience:** Enhances UX by providing immediate feedback and control.
 - **Interaction:** Enables communication between users and web applications.

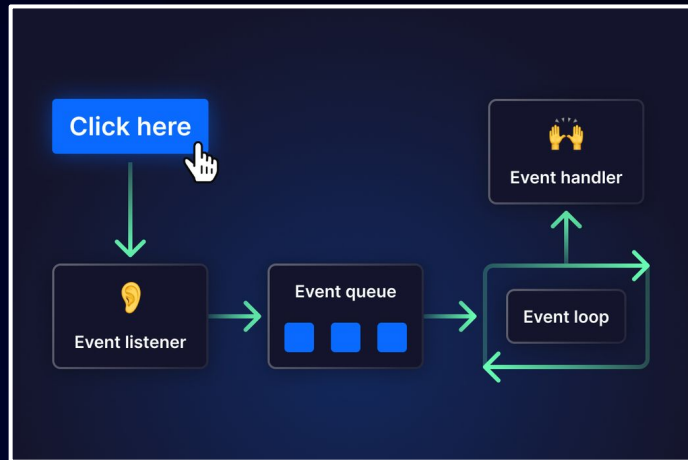
Types of JavaScript Events

User Interaction Events:

- **Click:** Triggered by mouse clicks.
- **Key Presses:** Triggered by keyboard presses.

System Events:

- **Loading:** Triggered when a page or resource is loaded.
- **Errors:** Triggered when there is an error in scripts or resources.



Pop Quiz

Q. “Click on a button” is ?

A

Not a JavaScript Event

B

JavaScript Event

Pop Quiz

Q. “Click on a button” is ?

A

Not a JavaScript Event

B

JavaScript Event



Adding and Removing Event Listeners

Adding Event Listeners

- **Method:** `addEventListener(event, handler)`
- **Syntax**

```
element.addEventListener('eventType', function(event) {  
    // Event handling code  
});
```

- **Example**

```
document.getElementById('myButton').addEventListener('click', function() {  
    console.log('Button Clicked');  
});
```

Removing Event Listeners

- **Method:** removeEventListener(event, handler)
- **Syntax**



```
element.removeEventListener('eventType', function(event) {  
});
```

- **Example**



```
function handleClick() {  
    console.log('Button Clicked');  
}  
  
var button = document.getElementById('myButton');  
button.addEventListener('click', handleClick);  
button.removeEventListener('click', handleClick);
```



Hands-On Demo | Event Listeners

Q.Which method is used to attach an event handler to an element?

A

addEventListener

B

addEventListener

Pop Quiz

Q.Which method is used to attach an event handler to an element?

A

addEventListener

B

addEventListener



Keyboard Events

Introduction to Keyboard Events

Definition: Keyboard events are triggered by keyboard actions such as pressing or releasing keys.

Types:

- keydown
- keyup
- keypress (Deprecated)



keydown Event

- Triggered when a key is pressed down.
- **Usage:** Useful for handling keyboard shortcuts or responding to key presses.
- **Example:**

```
document.addEventListener('keydown', function(event) {  
  console.log('Key Down: ', event.key);  
});
```

keyup Event

- Triggered when a key is released.
- **Usage:** Useful for actions that should occur after the key is released, such as form submission.
- **Example:**



```
document.addEventListener('keyup', function(event) {  
    console.log('Key Up: ', event.key);  
});
```

keypress Event

- Triggered when a key is pressed (deprecated, but still useful).
- **Usage:** Primarily used for capturing character input, though less recommended due to deprecation.
- **Example:**

```
document.addEventListener('keypress', function(event) {  
    console.log('Key Press: ', event.key);  
});
```



Hands-On Demo | Keyboard Events

Pop Quiz

Q.Which event is triggered when a key is pressed down?

A

keypress

B

keydown

Pop Quiz

Q.Which event is triggered when a key is pressed down?

A

keypress

B

keydown



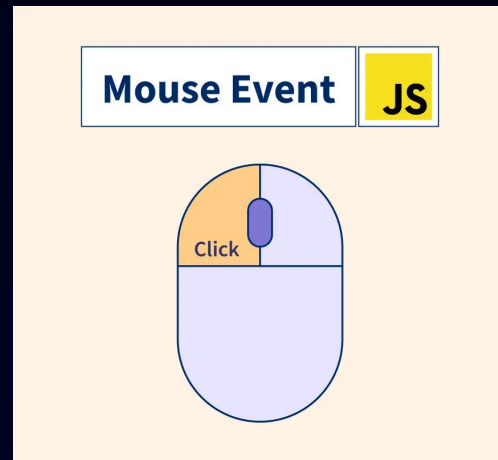
Mouse Events

Introduction to Mouse Events

Definition: Mouse events are triggered by mouse actions such as clicking or moving the cursor.

Types:

- click
- dblclick
- mouseover
- mouseout
- mousedown
- mouseup



click Event

- Triggered when an element is clicked.
- **Usage:** Commonly used for button clicks and link activations.
- **Example:**



```
document.getElementById('myButton').addEventListener('click', function() {  
    console.log('Button Clicked');  
});
```

dblclick Event

- Triggered when an element is double-clicked.
- **Usage:** Used for actions that should occur on a double-click, such as text selection.
- **Example:**



```
document.getElementById('myButton').addEventListener('dblclick', function() {  
    console.log('Button Double Clicked');  
});
```

mouseover Event

- Triggered when the mouse pointer moves over an element.
- **Usage:** Useful for displaying tooltips or changing element styles on hover.
- **Example:**



```
document.getElementById('myButton').addEventListener('mouseover', function() {  
    console.log('Mouse Over Button');  
});
```

mouseout Event

- Triggered when the mouse pointer moves out of an element.
- **Usage:** Useful for starting drag operations or initiating actions on button press.
- **Example:**

```
document.getElementById('myButton').addEventListener('mouseout', function() {  
    console.log('Mouse Out Button');  
});
```

mousedown Event

- Triggered when the mouse button is pressed down.
- **Usage:** Used to revert styles or hide tooltips when the mouse moves away.
- **Example:**

```
document.getElementById('myButton').addEventListener('mouseout', function() {  
    console.log('Mouse Out Button');  
});
```

mouseup Event

- Triggered when the mouse button is released.
- **Usage:** Often used in conjunction with mousedown for drag operations or form submissions. Often used in conjunction with mousedown for drag operations or form submissions.
- **Example:**

```
document.getElementById('myButton').addEventListener('mouseup', function() {  
  console.log('Mouse Button Up');  
});
```



Hands-On Demo | Mouse Methods

Q.Which mouse event is fired when the mouse button is released?

A

mouseup

B

click

Q.Which mouse event is fired when the mouse button is released?

A

mouseup

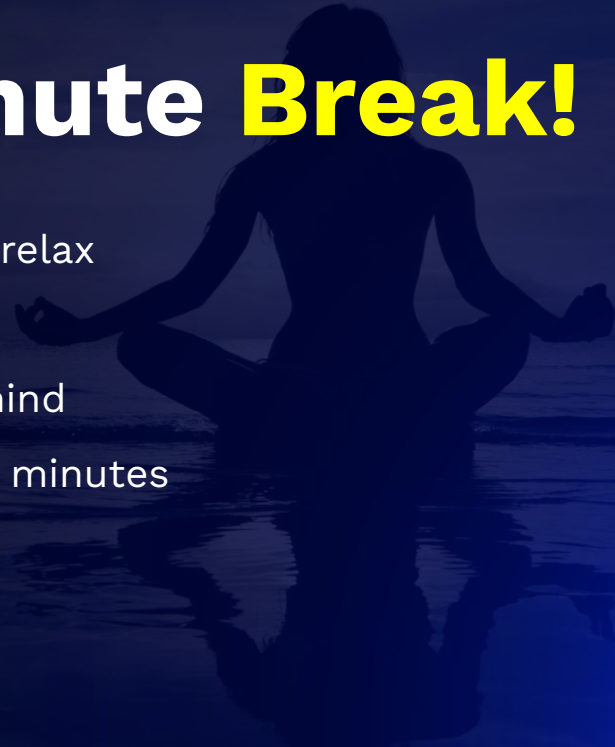
B

click



Take A 5-Minute **Break!**

- Stretch and relax
- Hydrate
- Clear your mind
- Be back in 5 minutes

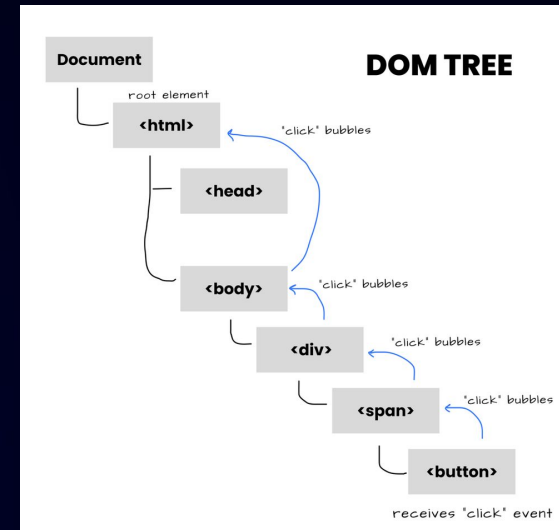




Event Propagation: Bubbling and Capturing

Introduction to Event Propagation

- **Definition:** Event propagation refers to the order in which events are received by the elements in the DOM hierarchy.
- **Importance:** Understanding propagation is crucial for effective event handling and management in complex web applications.



Bubbling Phase

- The event starts at the target element and bubbles up to the root of the DOM.

- **Example**

```
document.getElementById('child').addEventListener('click', function() {  
    console.log('Child element clicked');  
});  
  
document.getElementById('parent').addEventListener('click', function() {  
    console.log('Parent element clicked');  
});
```

- **Explanation:** Clicking the child element will trigger the event handler on the child first, then bubble up to the parent element.

Capturing Phase

- The event starts from the root of the DOM and captures down to the target element.

- **Example**

```
document.getElementById('parent').addEventListener('click', function() {
  console.log('Parent element clicked');
}, true); // Use capture phase

document.getElementById('child').addEventListener('click', function() {
  console.log('Child element clicked');
});
```

- **Explanation:** The event handler on the parent element is triggered first, before the child element, due to capturing phase.

Comparing Bubbling and Capturing

Feature	Event Bubbling	Event Capturing
Direction	Target → Parent → Document	Document → Parent → Target
Order	Bottom-up	Top-down
Phase	3rd phase	1st phase
Usage	Most commonly used	Rarely used
Trigger Flow	Starts from clicked element	Starts from root element



Hands-On Demo | Event Propagation

Pop Quiz

Q.Which phase starts at the root and captures down to the target element?

A

Bubbling

B

Capturing

Pop Quiz

Q.Which phase starts at the root and captures down to the target element?

A

Bubbling

B

Capturing



Event Delegation

Introduction to Event Delegation

Definition: Event delegation involves using a single event listener on a parent element to manage events for multiple child elements.

Benefits:

- **Efficiency:** Reduces the number of event listeners.
- **Dynamic Content:** Handles events for elements added dynamically.

```
<ul>
  <li>.....</li>  _____  event listener----- responding function
  <li>.....</li>  _____  event listener----- responding function
  <li>.....</li>  _____  event listener ----- responding function
  <li>.....</li>  _____  event listener ----- responding function
  <li>.....</li>  _____  event listener ----- responding function
  <li>.....</li>  _____  event listener ----- responding function
  ....
  ....
  ....
</ul>
```

Implementing Event Delegation

- Sets the value of an attribute on an element.
- **Example**

```
document.getElementById('parent').addEventListener('click', function(event) {  
    if (event.target && event.target.matches('button.child')) {  
        console.log('Child button clicked:', event.target.textContent);  
    }  
});
```

- The event listener on the parent element handles clicks for all buttons with the class child, even if they are added dynamically.



Hands-On Demo | Event Delegation

Pop Quiz

Q.What is a primary benefit of event delegation?

A

Increases event listener count

B

Handles dynamically added elements

Pop Quiz

Q.What is a primary benefit of event delegation?

A

Increases event listener count

B

Handles dynamically added elements



Preventing Default Behavior and Propagation

preventDefault Method

- **Description:** Stops the default action of an event (e.g., preventing form submission).
- **Example**

```
document.getElementById('myForm').addEventListener('submit', function(event) {  
    event.preventDefault();  
    console.log('Form submission prevented');  
});
```

- Prevents the form from submitting and allows custom handling of the form data.

stopPropagation Method

- **Description:** Stops an event from propagating further (bubbling or capturing).
- **Example**

```
document.getElementById('child').addEventListener('click', function(event) {  
    event.stopPropagation();  
    console.log('Child element clicked, propagation stopped');  
});  
  
document.getElementById('parent').addEventListener('click', function() {  
    console.log('Parent element clicked');  
});
```

- Clicking the child element will trigger only the child's event handler, stopping the event from reaching the parent.

stopImmediatePropagation Method

- **Description:** Stops the event from triggering any other listeners on the same element.
- **Example**

```
document.getElementById('myElement').addEventListener('click', function(event) {
  console.log('First handler');
  event.stopImmediatePropagation();
});

document.getElementById('myElement').addEventListener('click', function() {
  console.log('Second handler (will not run)');
});
```

- The second event handler will not run due to stopImmediatePropagation.



Hands-On Demo | Preventing Default Behaviour and Propagation

Pop Quiz

Q.Which method is used to prevent the default action of an event?

A

stopPropagation

B

preventDefault

Pop Quiz

Q.Which method is used to prevent the default action of an event?

A

stopPropagation

B

preventDefault





Guided Practice Tasks on Events & Interactive Applications

Summary

In this lecture, we have covered:

- Mouse and keyboard events handle user interactions like clicks and key presses.
- `addEventListener()` adds events, and `removeEventListener()` removes them.
- Events follow propagation: capturing (top-down) and bubbling (bottom-up).
- Event delegation uses a parent to handle child events efficiently.
- `preventDefault()` stops default browser actions.
- `stopPropagation()` stops event flow in the DOM.

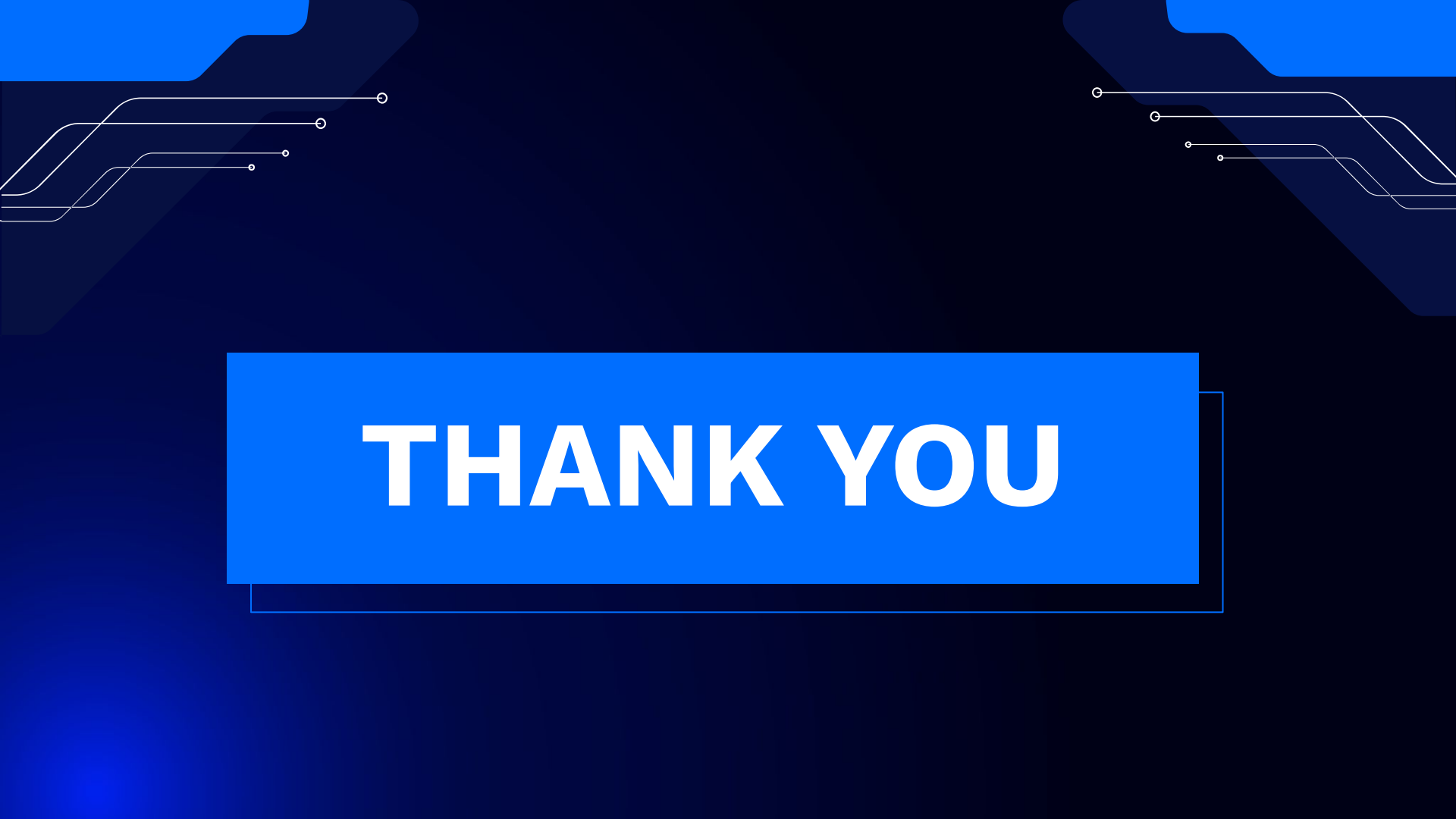




DOUBT RESOLUTION

Important

- Complete the post-class assessment
- Complete assignments (if any)
- Practice the concepts and techniques taught in this session
- Review your lecture notes
- Note down questions and queries regarding this session and consult the teaching assistants.



THANK YOU